

THE LIBRARY • THE 101S

AI Fundamentals 101

A plain-English map of the different types of AI, the parts behind the screen and how a model learns.

Understand what kind of AI you are looking at, what is doing the work and what changes when you use it.

INTRODUCTION THROUGH CHAPTER 3 • LOCAL REVIEW PROOF • NOT ADMITTED OR PUBLISHED

AI is not one thing, one company or one all-knowing brain in the sky. This is the plain-English map of what is actually happening—from the physical machines and model training to the information an AI product uses for your answer.

Contents

Introduction — Why is AI worth understanding if you are not a technologist?

How understanding the system helps you use it better, spot weak claims and take part in decisions without borrowing someone else's confidence.

Chapter 1 — What are the different types of AI—and why do the lists disagree?

The different jobs, media, methods and breadth people mean by *type of AI*—and why a chatbot, image generator and fraud detector can belong to several types at once.

Chapter 2 — What parts make an AI system work?

The product, model, instructions, information, tools, physical computers and people that turn a human request into a result or action.

Chapter 3 — How does a model learn from data—and produce a result?

The data, goals and feedback that shape a model; the difference between training and use; and why different models do different jobs.

Chapter 4 — What information does an AI use when you ask it something?

How your prompt, chat history, files, saved memory and retrieved information become—or fail to become—part of the current answer.

Chapter 5 — How does an AI product do more than answer?

Tools, APIs, connectors, permissions and multimodal generation—and the difference between a model suggesting an action and software carrying it out.

Chapter 6 — What is an AI agent—and how does it carry out a multi-step job?

Workflows, agents, plans, loops, observations, skills, stopping conditions and human checkpoints.

Chapter 7 — Why can AI be wrong, biased or unsafe?

Variation, hallucination, bias, unsuitable data or models, evaluation, benchmarks, reliability, fairness and safety as different questions.

Chapter 8 — What happens to your data—and who controls the system?

Collection, storage, retention, training use, access, security, ownership, accountability, correction and contestability.

Chapter 9 — How do AI systems change work, resources and public decisions?

Tasks and jobs, human-AI collaboration, labour, energy and materials, deployment, monitoring, governance and how to read a dated AI claim.

Concept Index for the beginning of this book

Use this when you meet a term and want to know where it fits. Each entry gives the shortest useful meaning and routes back to the connected explanation.

- **Accelerator chip:** hardware designed to perform demanding calculations efficiently; GPUs and TPUs are examples used in AI work. See [Chapter 2](#).
- **Agentic AI:** a system pattern that can choose actions, use tools, observe results and continue across steps toward a goal. It is not the same as generative AI or AGI. See [Chapter 1](#).
- **AGI:** artificial general intelligence, a disputed proposed category based on broad generality and transfer across unfamiliar intellectual tasks. LAiDIES does not treat current systems as AGI. See [Chapter 1](#).
- **AI system:** the complete working arrangement—people, product, models, information, tools, controls and physical infrastructure—not one magic object. See [Chapter 2](#).
- **Artificial intelligence (AI):** the broad field of building machine-based systems that produce outputs such as predictions, recommendations, content or decisions. See [Chapter 1](#).
- **ASI:** artificial superintelligence, a hypothetical category for capability beyond human or collective-human levels across a broad intellectual range. It does not exist today. See [Chapter 1](#).
- **Data centre:** a physical facility containing servers, storage, networks, power and cooling used to run computing work. See [Chapter 2](#).
- **Deep learning:** a type of machine learning using multilayer neural networks; it sits inside machine learning, not beside all of AI. See [Chapter 1](#).
- **Fine-tuning:** additional training that adjusts an existing model for a task or desired behaviour; it is different from giving the model a prompt or file for one use. See [Chapter 3](#).
- **Generative AI:** AI designed to produce new content such as text, images, audio, video or code. See [Chapter 1](#).
- **General-purpose AI:** a current system that can work across a wide range of tasks; this does not make it AGI. See [Chapter 1](#).
- **Generalisation:** using learned patterns successfully on appropriate new inputs rather than only repeating training examples. See [Chapter 3](#).
- **GPU:** a graphics processing unit; an accelerator whose parallel calculations are useful for many AI workloads. See [Chapter 2](#).
- **Inference:** using a trained model on new input to produce an output. See [Chapter 3](#).

- **Knowledge cutoff:** a simplified label for the time boundary of a model's training knowledge. Coverage is uneven; newer information can be supplied during use without retraining the model. See [Chapter 3](#).
 - **Language model:** a model that learns patterns in sequences of language. An LLM is a large language model, not a name for every kind of AI. See [Chapter 1](#) and [Chapter 3](#).
 - **Loss:** a numerical signal measuring how far a training output is from the objective being rewarded. See [Chapter 3](#).
 - **Machine learning:** a family of AI methods that learn patterns from data instead of relying only on hand-written rules. See [Chapter 1](#).
 - **Modality:** the kind of input or output a system works with, such as text, image, audio or video. See [Chapter 1](#).
 - **Model:** a learned mathematical structure and parameters that transform an input into an output; it is one component of an AI system. See [Chapter 2](#).
 - **Multimodal:** able to work across more than one modality, such as an image plus a written question. See [Chapter 1](#).
 - **Neural network:** a model structure made of connected layers of adjustable units. Modern deep-learning models use many layers. See [Chapter 3](#).
 - **Objective:** the result a training process is trying to improve. The objective helps shape what the model becomes good at. See [Chapter 3](#).
 - **Optimisation:** the repeated process of adjusting model parameters to reduce loss. See [Chapter 3](#).
 - **Parameters:** adjustable numerical values in a model that training changes. See [Chapter 3](#).
 - **Product or app:** the interface and surrounding software that choose which model, instructions, information, tools and controls a person encounters. See [Chapter 2](#).
 - **Server:** a computer that provides processing, storage or services to other devices or software. See [Chapter 2](#).
 - **Specialised AI:** a system built or configured for a bounded set of tasks. See [Chapter 1](#).
 - **Token:** a unit a language model processes, which may be a word, part of a word, punctuation or another small piece. See [Chapter 3](#).
 - **Tokenizer:** the process or component that converts text into tokens. See [Chapter 3](#).
 - **Training:** the process that uses data and an objective to adjust a model's parameters. See [Chapter 3](#).
 - **TPU:** Google's specialised tensor processing unit, one example of an accelerator used for machine-learning workloads. See [Chapter 2](#).
-

Introduction — Why is AI worth understanding if you are not a technologist?

AI can now enter a meeting before anyone has agreed what the words *use AI* mean.

One person means a chat product drafting the agenda. Another means software ranking job applicants. Someone else is proposing an automated customer-service workflow that can issue refunds. The slide still says **AI initiative**, as if these were one activity and one risk wearing a very confident blazer.

The same collapse happens outside work. A search result arrives with an AI-written summary. A bank, insurer or public agency announces an AI-assisted process. A friend shares a miraculous medical answer from a chatbot. A headline says AI will transform energy, jobs, education or science. Some of those claims describe real and important changes. Some skip three layers of mechanism and several inconvenient questions. Most use the same two letters.

You do not need a computer-science degree to tell the difference. You need a map.

Not a map of every company, product and model name. That would be stale before the ink dried. You need the stable parts: the physical equipment that makes computation possible; the model shaped during training; the product that decides what reaches it; the information, tools and permissions available for this task; the output that comes back; and the person or system deciding what happens next.

Once you can see those parts, AI stops being a magic word.

It also becomes more fun to explore. You can ask a language model to turn a brain-dump into a plan, use an image model to test a party-table idea before buying anything, let a transcription tool rescue the useful parts of a voice note, or compare three versions of an awkward email without believing that the machine has suddenly become your wisest friend. Understanding the mechanism does not spoil the trick. It gives you more ways to use it on purpose.

You can use it better because you know whether a weak result needs a clearer goal, missing information, a different tool or a different kind of model. You can challenge a workplace proposal without pretending to be an engineer: What job will the system do? What information will it use? What happens to the result? You can read a public claim about chips, energy, safety or jobs and ask which part of the system the claim actually describes.

Most importantly, you can keep your judgement in the room.

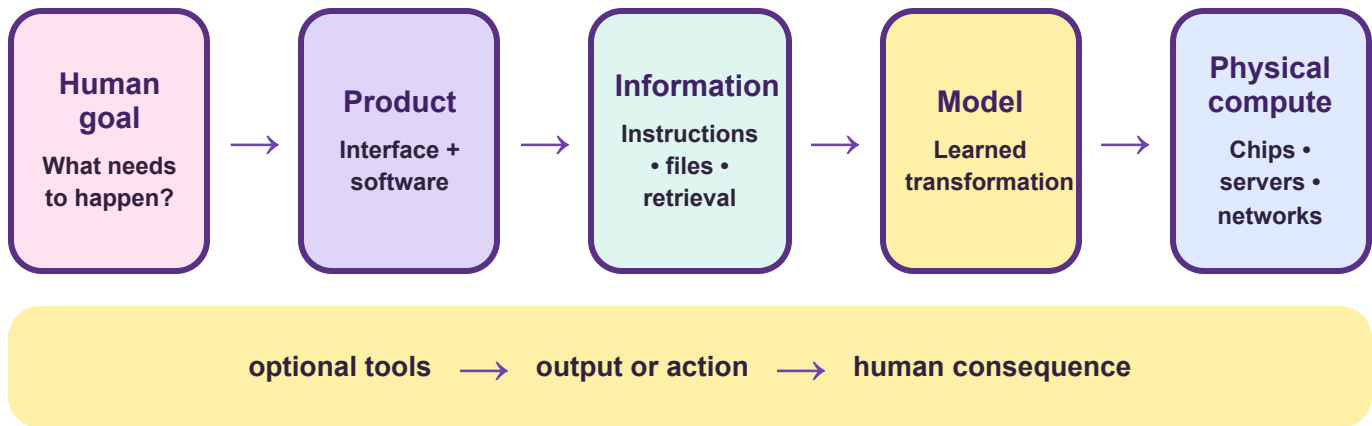
That does not mean hovering over every harmless draft as though it were a nuclear launch code. It means matching your attention to the consequence. A brainstorm can tolerate mess. A benefits decision, medical claim, employment recommendation or message sent in your name deserves much more care.

Before AI gets a seat at the table, find out who briefed it, what it was given and what it is allowed to do.

The map we will build

THE MAP IN ONE GLANCE

An AI result comes from a chain, not one magic object



A human goal passes through a product, supplied information and a model running on physical computers. Optional tools may add information or actions. A person or institution still decides what happens to the result.

This book follows one connected path:

human goal → product → supplied information → model → optional tools or retrieval → output or action → consequence

The exact arrangement varies. A small model may run on a phone. A product may call several models. A system may retrieve documents, use a calculator or take an action in another service. Some systems have no chat box at all. The map is not a claim that every AI product is identical. It is a way to locate the part that matters.

Chapter 1 sorts the different things people mean by *type of AI*. Chapter 2 shows the product, model, physical machines and people that make a complete AI system work. Chapter 3 explains how data and feedback shape a model and what changes when you use it.

Later chapters will add tools, permissions, workflows and agents; explain why results vary or fail; follow what happens to data; and widen the view to work, resources and public decisions. The Concept Index provides a direct route to one term without turning the book into a wall of definitions.

You are not here to memorize an org chart for machines. You are here to gain a working model of the system—strong enough to help you use it, question it and take part in what happens next.

Chapter 1 — What are the different types of AI—and why do the lists disagree?

Ask five people to name the types of AI and you may get five different lists.

One person says *machine learning, deep learning and generative AI*. Another says *text, image, audio and video*. A third says *predictive, generative and agentic*. Then somebody adds *narrow AI, AGI and ASI*, and the conversation begins to feel as though everyone arrived at the same party with a different dress code.

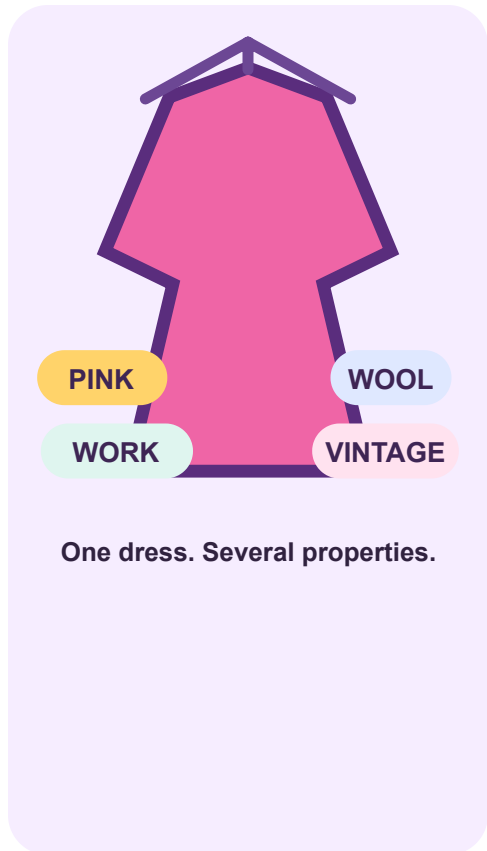
The lists disagree because they are answering different questions.

An AI system can be classified by the job it does, the material it works with, the way its model was built, the breadth of tasks it can handle and the way people can access it. One system can therefore belong to several types at once. That is not a contradiction. It is the beginning of a useful map.

One thing can carry several labels

CHAPTER 1 VISUAL

One product can carry several truthful labels



1 • Job

transcribe • classify • generate

2 • Modality

audio in • text out

3 • Method

machine learning • deep learning

4 • Breadth

general-purpose model in a specialist product

5 • Access

open • open weight • source available • closed

Agentic?

Only if it continues through actions.

The labels are not competing answers. They classify different properties. Agentic describes an operating pattern and can overlap several of them.

In the LAiDIES Closet, one dress can be black, wool, workwear, vintage and a size 10. If someone asks, “What type of dress is it?”, all five answers may be true. Colour, material, purpose, era and size describe different properties.

AI labels work in a similar way.

The analogy stops at classification. A model is not a dress, and its behaviour cannot be understood by reading a neat tag sewn into the lining. The useful idea is that a *type* only makes sense after you know which property is being classified.

Here are the five questions that organise this book.

1. What job does it do?

Some AI systems sort or estimate. Others create or act.

- A **classification** model assigns a category: likely fraud or not, urgent message or ordinary message, cat or dog.
- A **prediction** or forecasting model estimates an outcome or quantity: next month's demand, the probability of a missed payment, tomorrow's energy use.
- A **ranking** model puts options in order: which search result, applicant or product should appear first.
- A **recommendation** system selects items it predicts a person may find relevant: a film, song, article or next purchase.
- A **generative** system produces new content: text, images, audio, video or code.
- A **control** system helps choose an action in a changing environment: adjust a thermostat, guide a robot arm or support part of a driving system.

These jobs can be combined. A shopping app may classify products, rank search results, recommend alternatives and generate a summary on the same screen. “It uses AI” still does not tell you which operation affected what you saw.

Generative and agentic describe different things

Generative AI describes what the system produces. A language model can generate an email. An image model can generate a poster. A music model can generate an instrumental track that sounds suspiciously ready for a 1998 rom-com montage.

Here, **generation** is the content-producing job. **Generative AI** names systems designed to do that job.

A **language model** learns patterns in sequences of language. A **large language model**, or LLM, is a language model trained at large scale. It is one kind of generative model, not the name for every model or every AI system.

Agentic AI describes how a system operates across a job. An agentic system can work toward a goal by choosing an action, using a permitted tool, observing the result and deciding what to do next. Instead of answering once, it can continue through several steps.

Those are not rival categories.

- A chatbot that drafts one paragraph may be generative but not agentic.
- An agent that checks stock levels and reorders an approved item may be agentic while using classification, rules and tools as well as generation.
- A research assistant that searches, reads, compares and writes may be both agentic and generative.

Agentic does not mean **AGI**. A system can choose among tools inside a narrow workflow without possessing general intelligence. A self-checkout also moves through several steps; nobody has invited it to solve the mysteries of human existence.

Chapter 6 will explain the full agent loop after we have learned about models, current information and tools. For now, remember the distinction: *generative* describes producing content; *agentic* describes continuing through actions.

Keep the distinction

Generative tells you that a system produces content. **Agentic** tells you that a system can continue through actions toward a goal. **AGI** is a disputed proposed threshold for broad, transferable capability. None of these words is a synonym for either of the others.

2. What kind of material does it work with?

The technical word is **modality**: a kind of input or output.

A model may work with:

- text;
- images;
- speech or other audio;
- video;
- tables, measurements or other structured data; or
- signals from sensors and machines.

A **multimodal** model can work across more than one. For example, it might receive a photograph and a written question, then answer in text. A meeting assistant may take audio, turn it into text and produce a written summary.

Multimodal does not mean limitless. A system that accepts text and images may not accept video. A model that can describe a chart may still struggle to read a tiny label or preserve an exact number. The type tells you which forms it can process, not how well it will perform on every example.

3. How was it built?

Some software follows rules written directly by people: *if this condition is true, do that*. Rule-based systems are part of AI's history and remain useful when the rule is known and stability matters.

Machine learning is a family of methods in which a system uses data to adjust a model so it can find patterns rather than relying only on rules written in advance.

Deep learning is a machine-learning approach that uses neural networks with many layers. Most of the language, image, audio and video models behind the current generative-AI boom use deep learning.

So these labels describe a rough family relationship:

AI contains machine learning; machine learning contains deep learning.

Generative does not sit neatly underneath them as another building method. It describes the job of producing content. Most current generative models use deep learning, but many deep-learning models classify, rank or predict instead.

Chapter 3 will show how data and feedback shape a machine-learning model. At this point, you only need the branches to stop the words collapsing into one another.

4. How broad is its capability?

A **specialised** system is built or configured for a bounded set of tasks. A fraud detector may be excellent at finding transaction patterns and useless at planning a holiday.

A **general-purpose** model can work across a much wider range of tasks. Modern language and multimodal models can draft, summarize, translate, analyse images, help with code and move among subjects. That breadth is real. It also blurs the old habit of describing every present system as able to do only one narrow thing.

General-purpose does not mean *general intelligence*. The first describes current systems that can handle many tasks. The second appears in the disputed proposed category AGI.

General-purpose is not the same as **artificial general intelligence**, or **AGI**.

AGI is a proposed and disputed threshold. Definitions vary, but an important idea is broad generality: the ability to transfer capability across unfamiliar intellectual tasks rather than performing well only on a known collection of tasks. There is no universally agreed definition or test. Credible experts disagree about what would count and whether or when it might happen. LAiDIES does not treat any system available today as AGI.

Artificial superintelligence, or **ASI**, is more speculative still. It describes a hypothetical system whose intellectual capability would exceed human—or, in some definitions, collective human—capability across a very

broad range. ASI does not exist today. Proposed paths from AGI to ASI are research scenarios with major uncertainties, not a timetable for an inevitable product launch.

This gives us a boundary worth keeping:

- specialised and general-purpose describe systems that exist now;
- AGI is a disputed proposed category that LAiDIES says has not been reached;
- ASI is a hypothetical beyond-human category; and
- an agent using several specialist tools is not secretly AGI wearing a busy lanyard.

Chapter 9 will return to AGI and ASI when we learn how to examine public claims and forecasts without turning either enthusiasm or fear into evidence.

5. How can people access or change it?

You may also hear **open source**, **open weight**, **source available** and **closed** described as AI types. These labels answer another question: what parts of a model or system are available to inspect, download, change or use under particular terms?

They do not tell you whether the model generates or predicts, whether it is agentic, how broad it is, or whether it is accurate or safe. Two systems can do similar jobs while offering very different access. The exact distinctions matter enough to receive their own treatment in Chapter 8, alongside control, security and accountability.

For now, do not use *open* as a synonym for good or *closed* as a synonym for capable. Access is one property, not a verdict.

Pause and place it: put one product on the map

Consider a meeting assistant that records a discussion, creates a transcript, finds the decisions and prepares follow-up tasks.

What type of AI is it?

- **By job:** it may transcribe, classify, rank and generate.
- **By modality:** it moves from audio to text.
- **By method:** its main models likely use machine learning and deep learning.
- **By operating pattern:** if it only returns a summary, it may not be agentic; if it creates tasks, checks calendars and follows up across steps, it may use an agentic workflow.

- **By breadth:** it may use a general-purpose language model inside a specialised meeting product. It is not AGI.
- **By access:** its models and surrounding software may be open, open weight, source available or closed in different combinations.

No single label describes the whole product. The five axes tell you which question each label answers.

The first click

There is no one correct list of AI types because *type* can describe a job, modality, building method, breadth or form of access.

That gives you a useful reply the next time somebody says, “This is a different kind of AI.”

Different in what way?

The answer may be generative, agentic, multimodal, deep-learning, general-purpose or open weight—and several may be true at once.

Now we can ask the next question. Once we know what property a label describes, what parts have to work together to make the real system run?

Source notes

- [OECD.AI — updated AI system definition and application areas](#)
- [Google for Developers — Machine Learning Glossary](#)
- [Google DeepMind — Levels of AGI](#)
- [Google DeepMind — From AGI to ASI](#)

Concept boundaries checked 8 August 2026. AGI and ASI definitions, tests, capability claims and timelines must remain attributed, disputed and current.

Chapter 2 — What parts make an AI system work?

You open an app, type a request and watch an answer appear. The experience is so smooth that the app, the model and the whole AI system can feel like one thing.

They are not.

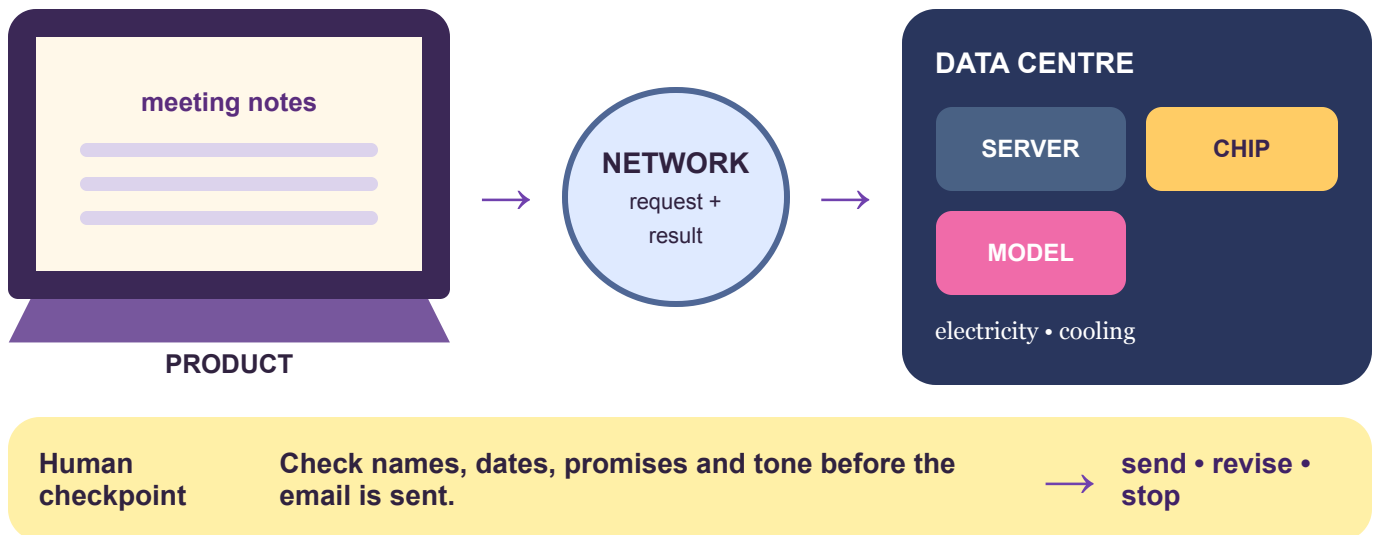
That distinction matters for the same reason it mattered when many of us first went online. Internet Explorer was not *the internet*. It was the doorway a lot of Rewind Era households used to reach it. Behind the blue **e** sat networks, servers, websites, companies, rules and a modem making a noise like two fax machines ending a friendship.

An AI app is also a doorway. The comparison stops there: AI systems are not the same architecture as the internet, and the analogy cannot explain training or generation. It does one useful job. It reminds us that the thing on the screen is not the whole thing.

Start with the whole system

CHAPTER 2 VISUAL

The app is the doorway, not the whole AI system



The product assembles working input. Computing hardware runs the model. The product presents the result. Permissions and people determine whether it becomes an action.

An **AI system** is a machine-based arrangement that receives input and produces an output such as content, a prediction, a recommendation or a decision. That output may influence something in a digital environment or the physical world.

The system can include:

- the product or interface a person uses;
- one or more models;
- instructions and other information supplied for the task;
- software that routes requests or calls other services;
- search, retrieval, calculation or action tools;
- permissions and safety controls;
- computing hardware, networks, electricity and cooling; and
- people and organizations deciding where the output goes and what it is allowed to affect.

Not every system contains every item. The point is that **AI system** names the working arrangement, not one invisible brain.

The **product** is the part you encounter: the chat app, writing assistant, recommendation feature, fraud detector, image generator or software inside a work process. A product decides what the interface shows, which model or models it uses, which instructions and files it supplies, whether it can search or act, and what controls are available.

The **model** is one component inside that larger arrangement. In machine learning, a model is the learned mathematical structure and set of adjustable values—usually called parameters—that transform an input into an output.

That definition is less glamorous than “digital brain,” which is useful. A model has no little filing cabinet marked *everything you have ever told me*. It performs learned transformations on the input it receives. Depending on the model, the output might be generated text, a category, a risk score, a ranked list, a forecast, a detected object or a suggested action.

A company may build models, operate products, supply computing infrastructure or do all three. Brand names do not tell you which layer is doing the work.

Three levels that should not collapse

The **app or product** is what you encounter. The **model** is a learned component it may use. The **AI system** is the full working arrangement, including information, software, infrastructure, controls and people.

The cloud has a postcode

The word *cloud* makes computation sound as though it happens in a tasteful mist above our heads. It does not. The cloud has a postcode. It also has an electricity bill.

Much of today's model training and use happens in data centres: physical facilities containing rows of servers, storage systems and networking equipment, plus the machinery needed to power and cool them. A server is a computer built to provide processing, storage or services. Inside servers, general-purpose processors and specialized accelerator chips can perform the large volumes of mathematical operations used by AI workloads.

You will often hear **GPU**, short for graphics processing unit. GPUs were developed for graphics but are also good at performing many calculations in parallel. That makes them useful for many machine-learning jobs. **TPUs** are Google's specialized accelerators for machine-learning workloads. Other accelerators exist, and ordinary CPUs still do important work. "AI chip" is a category, not one universal object.

Large models may be too large or demanding for one processor. Their work can be divided across multiple chips and machines connected by high-speed networks. The computers need electricity. Their operation produces heat. Cooling systems move that heat away so the equipment can continue working. Networks move data between machines and carry requests and results between a product and the infrastructure behind it.

Some models run partly or entirely on a laptop or phone. Some products send a request to remote servers. Many systems combine local and remote work. "AI runs in data centres" is therefore a common pattern, not a rule that every keystroke must cross a continent.

What happens after you press Send

Imagine you ask a workplace assistant to turn a page of meeting notes into a clear follow-up email.

Here is the useful version of the journey:

1. **You define the job.** Your request and notes enter the product.
2. **The product assembles the working input.** It may add instructions, selected conversation history, files or other permitted information.

- 3. The request reaches computing hardware.** It may travel across a network to servers or be handled partly on your device.
- 4. A model processes the supplied input.** The model transforms that input into an output—in this case, generated language.
- 5. The product presents the result.** It may format the answer, attach source links or offer another action.
- 6. A person decides what happens next.** You check the names, promises, dates and tone before anything is sent in your name.

The polished email is not evidence that all six stages worked well.

If a colleague's decision was missing from the notes, the model could not recover it by confidence alone. If the product supplied an old version of the notes, the model worked from old information. If an email tool has permission to send automatically, a weak output can become an action before anyone checks it. If the email is accurate but unkind, the remaining problem is not compute. It is judgement.

This is why “the AI got it wrong” is often the beginning of the diagnosis, not the diagnosis itself.

The parts create different questions

Once the system is visible, broad claims become easier to examine.

If someone says, “AI uses a lot of electricity,” ask: training or everyday use? Which kind of system and hardware? Where is the data centre? What time period and comparison does the evidence cover?

If someone says, “The model has access to our policies,” ask: which product is using which model? How are the policies retrieved? Which versions can it reach? Are permissions enforced? Can the user inspect the source behind the answer?

If someone says, “AI approved the claim,” ask: did a model generate a score, a product apply a rule or a person make a decision after seeing an AI-assisted recommendation? Who is responsible for correcting an error?

Those are not technical gotchas. They are ordinary questions made more precise by knowing the parts.

Pause and place it: the answer was right, but the action was wrong

Suppose the meeting assistant drafts an accurate email and sends it to a client without the promised human check. Did the model fail to generate the email? Not necessarily. The defect sits in the surrounding workflow, permission or checkpoint. A good output can still be used badly by the larger system.

The second click

AI is not a single object inside your screen. It is a chain that begins in the physical world, includes a model and surrounding software, and ends in another human or institutional decision about what the output is allowed to do.

The app is what you open. The model is a learned component. The system is the whole working arrangement.

That is the foundation for Chapter 3. Now that **model** has a real place in the map, we can ask how one is made—and what changes when you use it.

Source notes

- [OECD.AI — updated AI system definition](#)
- [NIST — AI Risk Management Framework 1.0](#)
- [International Energy Agency — Energy demand from AI](#)
- [Stanford CS336 — Language Modeling from Scratch](#)
- [Google for Developers — Machine Learning Glossary](#)

Physical mechanisms checked 8 August 2026. Recheck any named hardware, facility, provider or numerical energy claim before publication.

Chapter 3 — How does a model learn from data—and produce a result?

A model is not born knowing how to classify a transaction, recognize speech or continue a sentence. Its useful behaviour is shaped through a process called **training**.

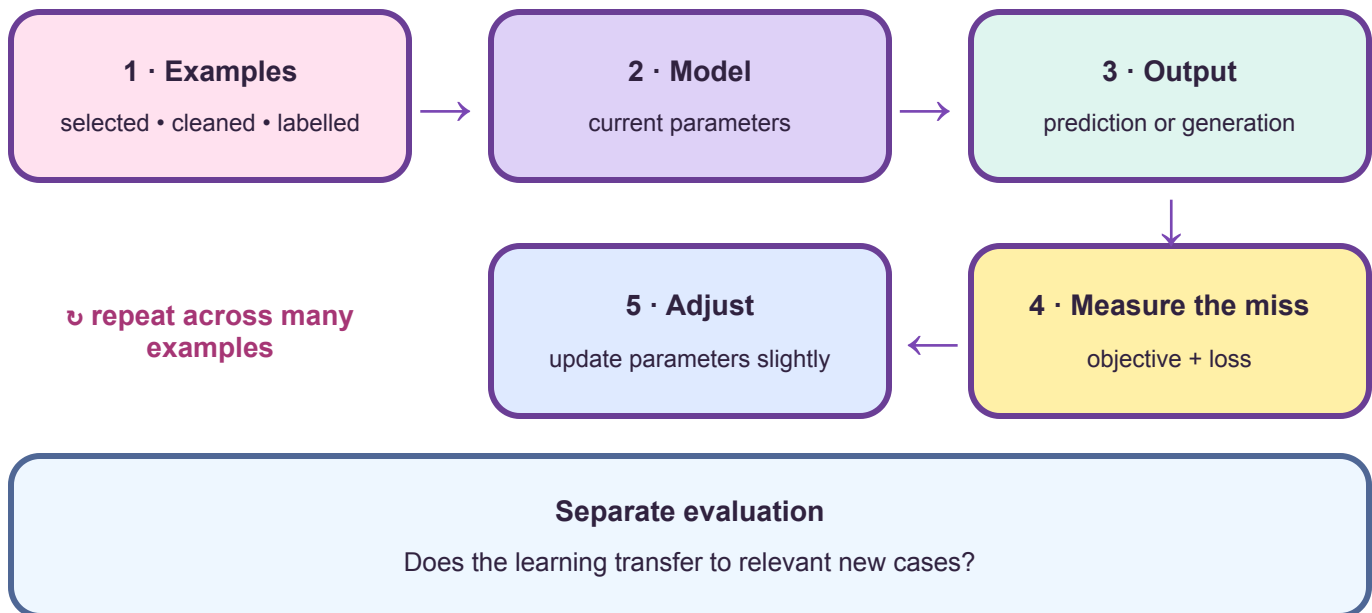
Training is different from using the model after it has been built. That sounds like a minor distinction until a product tells you it can “learn from your business” and everyone in the meeting nods at a completely different meaning of *learn*.

So let us separate the jobs.

What shapes a model during training

CHAPTER 3 VISUAL

Training adjusts a model through repeated measured misses



Examples and an objective shape the model through repeated parameter updates. Separate evaluation asks whether the learning transfers to relevant cases not used for that update.

During machine-learning training, a system gives the model many examples and a way to measure error against an objective. The training process adjusts the model's parameters so that its future outputs better fit that objective.

The exact method depends on the model. A simple fraud model might learn from past transactions labelled fraudulent or legitimate. An image model may learn patterns connecting images and descriptions. A language model learns patterns across sequences of tokens. Some systems also go through additional fine-tuning, human-feedback or reinforcement-learning stages to shape behaviour for particular tasks or product expectations.

Many modern models use a **neural network**: layers of connected mathematical units whose adjustable values transform an input step by step. *Deep learning* means using neural networks with many such layers. You do not need to calculate those layers to understand the important relationship: the architecture defines the paths available, while training adjusts the parameters along those paths.

Five choices do much of the shaping:

- **Data**: which examples are included, excluded, cleaned, labelled or mixed.

- **Representation:** how the material is converted into numbers the model can process, such as tokens for language.
- **Objective and loss:** what result the training rewards and how a miss is measured.
- **Optimisation:** how the model's parameters are adjusted to reduce that loss.
- **Evaluation:** which separate cases are used to test whether the learning travels beyond the training examples.

Then the training loop looks like this:

1. **Prepare examples.** People and systems collect, clean, filter, label or otherwise organize training material.
2. **Run the model on a training example.** The model produces a prediction or other output from its current parameter settings.
3. **Measure the miss.** A loss function turns the difference between the model's output and the training objective into a signal.
4. **Adjust the parameters.** An optimization process changes many adjustable values slightly in a direction intended to reduce that loss.
5. **Repeat.** Across many examples and iterations, the model becomes better at the patterns rewarded by the objective.
6. **Evaluate.** Developers test performance on material or tasks that were not used for the parameter update and examine where the model still fails.
7. **Prepare it for use.** A selected version can be saved, further shaped and made available for inference inside a product.

Parameters are not tidy drawers of source sentences. They are numerical values inside the model's learned structure. Training changes those values across many examples. The resulting model can produce useful outputs for new inputs without retrieving one stored answer for every possible question.

That is also why training data matters without acting like a searchable archive. The data influences the patterns a model can learn. Its selection, quality, gaps and errors affect performance. But a model answer usually cannot show you which training example “contains” each sentence, because generation is not a normal database lookup.

Learning a pattern is not the same as memorising the examples

The point of training is not merely to reproduce the practice material. A useful model must **generalise**: apply what it learned to appropriate new inputs.

Imagine teaching a new colleague to recognise expense claims that need a closer look. If she can only flag the six examples used in training, she has memorised the worksheet. If she can recognise the same warning patterns in a

new claim, she has learned something transferable. If she begins flagging every dinner receipt because two bad examples involved restaurants, she has learned the wrong shortcut.

Models can fail in similar ways. They may fit training examples closely but perform poorly on new cases, or learn correlations that do not hold in the setting where the product is used. That is why evaluation needs genuinely separate, relevant cases—not a victory lap over the same material used to train the model.

What tokens are doing here

Language models do not receive a sentence as one intact human idea. A **tokenizer** converts text into tokens the model can process. A token might be a whole common word, part of a word, punctuation or another small unit. The exact split depends on the tokenizer.

That is why 500 words and 500 tokens are not the same measurement, and why an unusual name may take more token space than a familiar word of similar length. For multimodal models, other input types can be converted into their own model representations—for example, pieces of an image rather than written words.

Tokens matter because they are the working units a language model is trained on and uses during generation. They also help determine how much material fits inside a model’s context window, which we will open properly in Chapter 4.

Why “it predicts the next token” is true but incomplete

An autoregressive language model generates text by repeatedly predicting a likely next token from the tokens that came before it.

That sentence often lands badly in one of two directions.

The first is hype: “It is reasoning exactly like a person.” The next-token mechanism does not establish human understanding, consciousness, lived experience or responsibility.

The second is dismissal: “So it is just phone autocomplete.” Also wrong. Training a large model across extensive data and many parameters can produce rich internal representations of language and relationships. Repeating the prediction step across a long context can generate plans, explanations, code and other complex structures. The mechanism is still next-token prediction; the learned system doing it is far more capable than the suggestion strip on a 2007 phone.

The training objective rewards a fitting continuation. That explains why a language model can produce a remarkably useful answer and still produce a confident mistake. Chapter 7 will deal with how to judge important results; here, the point is simply what generation is doing.

Inference uses the trained model

KEEP THE DISTINCTION

Training changes the model; inference uses it

Training / fine-tuning

1 · many examples → 2 · model → 3 · parameters change

Happens before this model version is used.

Inference / current use

1 · current input → 2 · model → 3 · new output

Parameters normally stay the same.

Training or fine-tuning changes parameters. During inference, a trained model processes current input. A prompt, file or retrieved source can change this result without automatically changing the underlying parameters.

When you type a new request into a product, the model is normally doing **inference**: using its already trained parameters to produce an output from new input.

Training changes the model. Inference uses the model.

Your current prompt can strongly change the current output because it changes the input. That is not the same as retraining the model's parameters. If you give a writing product three examples of your preferred tone, the model may use those examples in the current context. It has not necessarily absorbed your style into its general training for everyone forever.

Whether a provider stores your input or later uses product data to improve a model is a separate policy question. The answer can vary by provider, product, plan, account type and setting. Do not try to answer a data-governance question with the word *training* alone.

Training knowledge has a time boundary

Training has to stop before a particular model version can be prepared and used. Its learned parameters therefore reflect data and training completed before that version was produced.

Providers may publish a **training-data cutoff** or a more conservative **reliable-knowledge cutoff** for a named model. The useful idea is a time boundary, not a perfectly clean wall. A model may be uneven on events before the date, and training material can cover different sources and periods.

A Rewind Era household encyclopedia makes the distinction visible. The 1998 edition could contain an enormous amount of useful knowledge and still know nothing about an event in 2004. You could place a current newspaper beside it without reprinting the encyclopedia.

The analogy stops there. A model is not a set of reference volumes, its learned knowledge is not organised as readable articles, and a provider's cutoff does not promise complete recall before one date and total ignorance after it.

For an AI product, a newer fact can be supplied in the current prompt, an attached file, retrieved material or a live tool. That can help with this use without changing the model's trained parameters. Chapter 4 will explain those information paths.

Fine-tuning changes the model; prompting changes this use

After broad training, developers may use **fine-tuning** or other post-training methods to adjust a model for particular tasks or behaviour. That process changes parameters. It may help a general language model follow instructions, work in a specialist domain or behave more usefully inside a product.

A prompt, attached file or retrieved document usually does a different job. It adds information or direction for the current inference. It can change the answer dramatically without rewriting the trained model underneath.

This distinction matters when a product promises to be “trained on your business.” That phrase might mean real fine-tuning, retrieval from company documents, instructions inserted into every request—or an imprecise mixture of all three. The useful question is: **what changes in the model, and what is only being supplied for each use?**

One result, located properly

Return to the follow-up email from Chapter 2.

Long before your meeting, a language model was trained across many examples. That process adjusted its parameters and shaped its ability to work with language. Later, the model was evaluated, prepared for use and placed inside a product.

When you supplied your notes, the product assembled input for this task. The trained model processed those tokens during inference and generated the email token by token. The output reflected patterns learned during training and the specific material available now.

The model did not retrain itself on your meeting while you watched. It did not retrieve missing decisions from a mystical archive of office life. It produced language from its trained parameters and current input.

Now the distinction can do some work:

- If the model consistently performs poorly across many well-specified cases, the model, training or evaluation may be a poor fit.
- If the same model succeeds after you provide the missing notes, the first failure was at least partly an input problem.
- If the output needs a current policy the model was never given, the product may need retrieval or another source.
- If the generated email makes a promise nobody authorised, the remaining problem is the workflow and human decision—not vocabulary.

The third click

Training is how a system shapes a model. Data, representation, objectives, optimisation and evaluation all influence what it learns. Inference is what happens when that trained model processes new input. Tokens are the units a language model works with. Fine-tuning changes a model; prompting changes the current use.

The next question is unavoidable: what, exactly, counts as the input for your current request?

You may have typed it. The product may have saved it. The chat may still show it. A document may exist in your account. None of those facts alone proves the model received it for this answer.

That is Chapter 4.

Source notes

- [NIST — training stage glossary](#)
- [Stanford CS336 — Language Modeling from Scratch](#)
- [Google for Developers — Machine Learning Glossary](#)
- [Anthropic — model knowledge-cutoff definitions](#)
- [OECD.AI — updated AI system definition and application areas](#)

Mechanisms checked 8 August 2026. Recheck any named model's training data, architecture, tokenization, evaluation or provider data-use policy before publication.